

Lecture 25

Multithreaded Programming

CS 12 Team 25.2
A.Y. 2025–2026, 2nd Sem
Department of Computer Science
College of Engineering
University of the Philippines Diliman

Sanity Check

- Attendance?
- Audible at the back?
- TV(s) on?
 - (we're working on getting the other TV fixed...)
- Recording?
- Important announcements?

Multithreaded Programming

Motivating Problem

Task: Generate $n = 1,000,000,000$ random integers, store them into an array, and tally the number of positives, negatives, and zeros as fast as possible

25/random_integers.c

Motivating Problem

How can we make this faster?

Concurrency vs. Parallelism

Concurrency: Managing multiple tasks in a given span of time (may do interleaving)

- Chatting with different friends “at the same time”
- Event loop cycling through tasks, one at a time

Parallelism: Doing multiple tasks at the same time

- One representative assigned to each customer
- Multithreading on multicore processor

Threads

Thread: Abstraction for “current instruction” + call stack + environment

- “Thread of execution”
- Simplification: Arrow pointing to line of code + data
- Runs on a single CPU core (people doing work; multitasking myth)
 - Multicore processor = Multiple threads running in parallel
 - More in CS 21/140

Threads

- Programs are *single-threaded* by default
 - Runs on a single core of a multicore processor

How do we make multithreaded programs?

Multithreaded Programs and Concurrency

Shared-state concurrency: Threads communicate via shared memory

- C: Pthreads library
 - `#include <pthread.h>`
 - `gcc main.c -pthread -o main`
- Python: threading library ([docs](#))

Multithreaded Programs and Concurrency

Message-passing concurrency: Threads communicate via messages

- Out of scope of CS 12 (25.2)

Creating threads

```
int pthread_create(p_tid, attrs, function, arg)
```

- Creates new thread that runs `function`
- Returns 0 if successful

Creating threads

- `pthread_t *p_tid`
 - Pointer to thread ID of created thread
 - Output parameter; will be initialized
- `pthread_attr_t *attrs`
 - Set to NULL for CS 12
 - For the curious: [link](#)

Creating threads

- `void *(*function)(void *)`
 - Function to be executed by thread
 - Takes in `void *`, returns `void *`
 - Return value discussed later
- `void *arg`
 - Argument given to function

Creating threads

All programs start with a single thread (main thread)

25/create_thread.c

Awaiting thread termination

Execution order of threads on a multicore system is *nondeterministic*

```
int pthread_join(tid, returned_ptr)
```

- Causes invoking thread to *wait* for thread to finish
 - Used to make execution order deterministic
- Returns 0 if successful
 - ptr is set to void * returned by function

Awaiting thread termination

```
int pthread_join(tid, returned_ptr)
```

- `pthread_t tid`
 - Thread ID; not a pointer
- `void *returned_ptr`
 - Set to NULL if needed

Awaiting thread termination

25/await_thread.c

Race conditions

Race condition: Behavior depends on nondeterministic order of thread execution

Critical section: Region of code that could exhibit a race condition

Mutual exclusion: At most one thread is executing a critical section at any time

Race conditions

25/race_condition.c

How do we ensure race conditions do not happen?

Synchronization via locks

Lock/Mutex: Synchronization primitive that enforces mutual exclusion

- Lock can only be acquired if free/released (initial)
- Only one thread can acquire lock (how in CS 140)
- If lock is held, trying to acquire will make thread wait

Synchronization via locks

- `pthread_mutex_t`: Data type of lock
- `pthread_mutex_init(&lock, NULL)`: Initializes lock
- `pthread_mutex_lock(&lock)`: Acquires lock
- `pthread_mutex_unlock(&lock)`: Releases lock

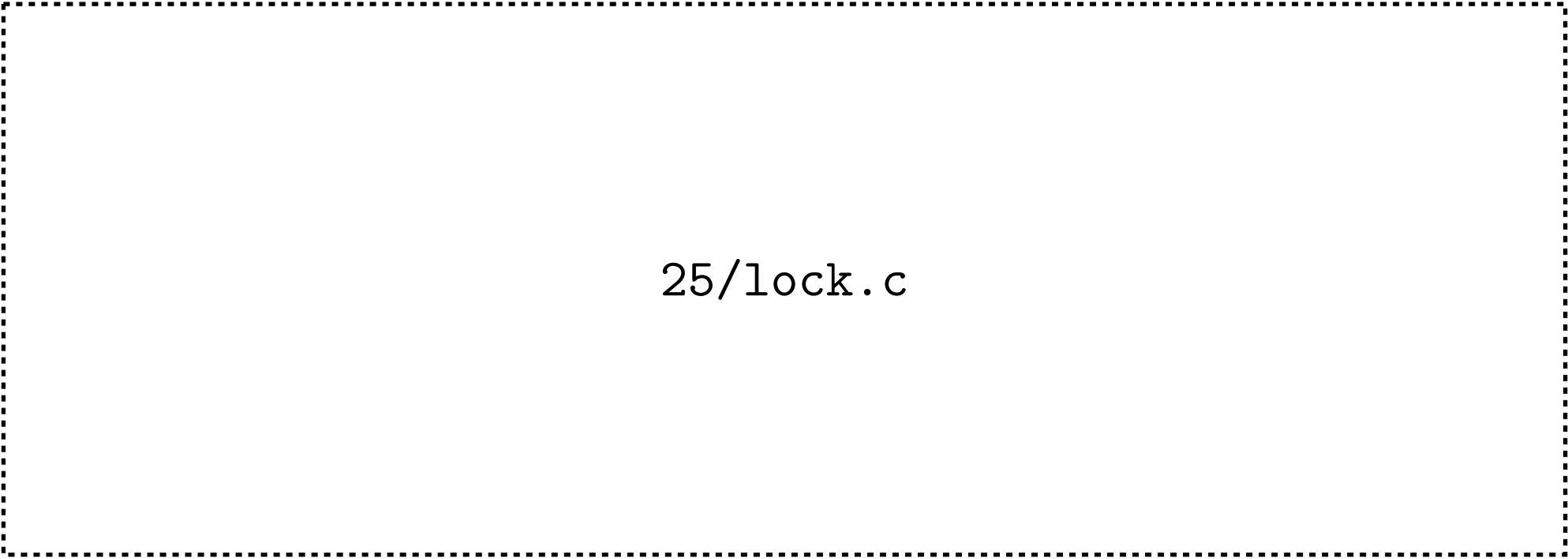
Synchronization via locks

```
// Self-check: Why is this worse than the given?
void *plus(void *arg) {
    pthread_mutex_lock(&lock);
    for (int i = 0; i < 10000; i++) {
        n++;
    }
    pthread_mutex_unlock(&lock);

    return NULL;
}
```

Synchronization via locks

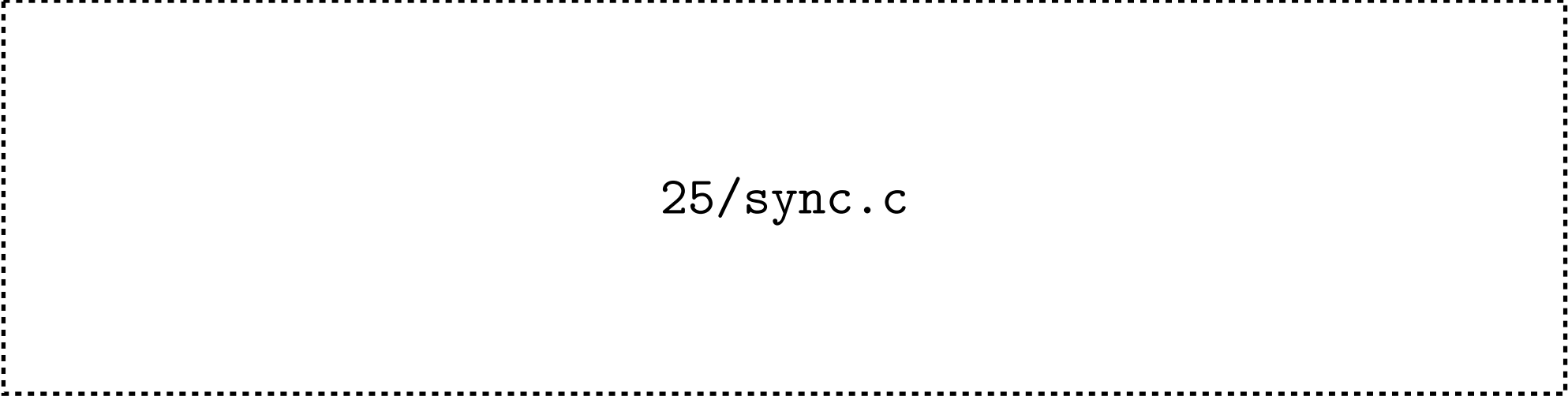
Same lock is used for accessing same resource



25/lock.c

Back to motivating problem

Idea: Split work into mutually exclusive chunks, distribute chunks to threads, then aggregate results to shared resource using lock



25/sync.c

Back to motivating problem

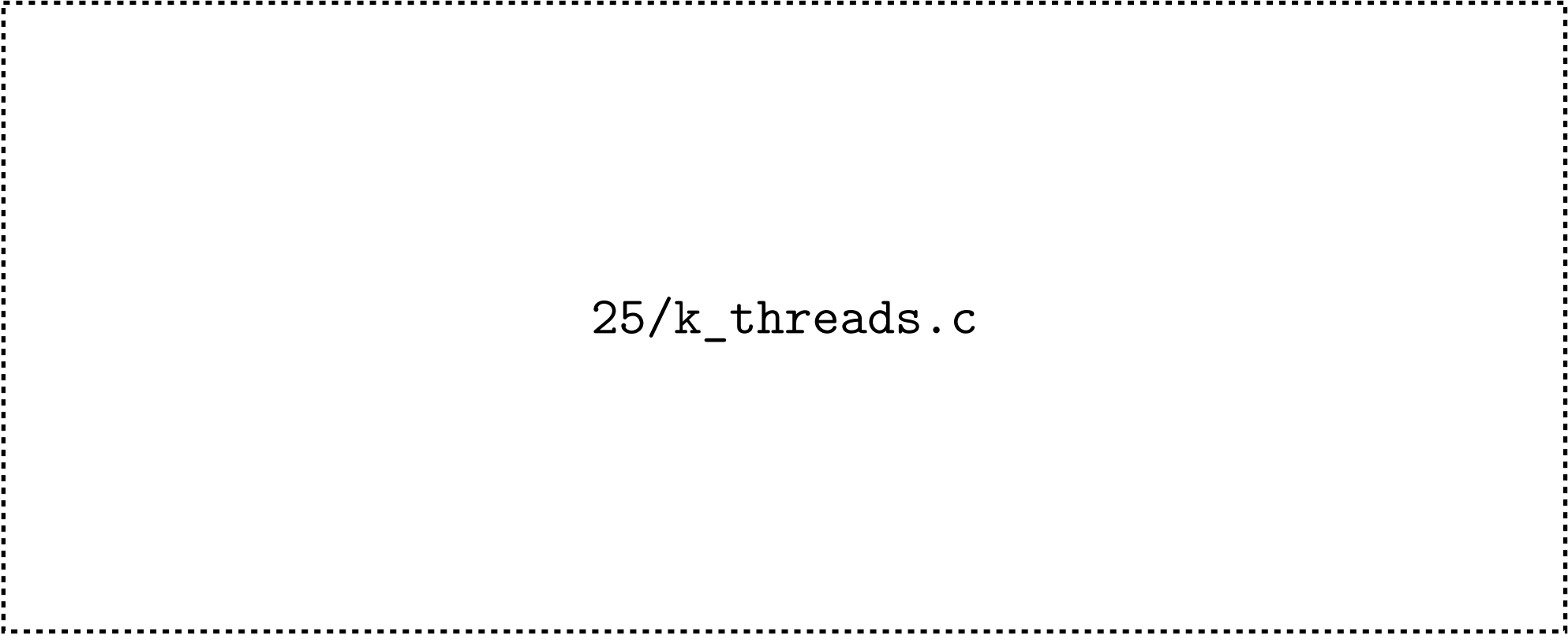
Why is this two-thread solution slower than with just a single thread?

Synchronization overhead

Locking and unlocking introduce additional overhead; better to minimize number of locking/unlocking operations

25/two_threads.c

Synchronization overhead



25/k_threads.c